

OSX Capital Distributor

Aragon

HALBORN

OSX Capital Distributor - Aragon

Prepared by: **HALBORN**

Last Updated 10/28/2025

Date of Engagement: September 9th, 2025 - September 15th, 2025

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED



TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
3. Test approach and methodology	
4. Risk methodology	
5. Scope	
6. Assessment summary & findings overview	
7. Findings & Tech Details	
7.1 Plugin address in strategy deployment breaks merkle root updates	
7.2 Pre-deployed strategy ownership mismatch breaks campaign creation	
7.3 Front-running strategy deployment causes campaign creation dos	
7.4 Campaigns can be created with end time in the past	
7.5 Fund lock due to campaign state management	
7.6 Malicious action encoder registration enables token theft	
7.7 Insufficient token validation in campaign creation	
7.8 Unused custom error daomismatch	
7.9 Unchecked array iteration in batchclaimcampaignpayout can cause dos	
7.10 Missing vault interface validation causes silent claim failures	
7.11 Missing metadata validation in registeractionencoder and registerstrategytype	

1. Introduction

Aragon engaged **Halborn** to conduct a security assessment on their smart contracts beginning on September 9th, 2025 and ending on September 15th, 2025. The scope of this assessment was limited to the smart contracts provided to the Halborn team. Commit hashes and additional details are documented in the Scope section of this report.

Capital Distributor is a modular Aragon OSx plugin that enables DAOs to create and manage onchain token distribution campaigns for purposes like airdrops, grants, incentives, and rewards. It supports customizable allocation strategies (e.g., Merkle trees) and payout methods (e.g., direct ERC20 transfers or vault deposits) through a factory-based architecture that deploys specific strategy and encoder contracts per campaign. The system allows full control over the campaign lifecycle including creation, pausing, and ending and integrates with Aragon’s permission system for secure access management.

2. Assessment Summary

Halborn was provided 5 days for the engagement and assigned 1 full-time security engineer to review the security of the smart contracts in scope. The engineer is a blockchain and smart contract security expert with advanced penetration testing and smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the smart contracts.
- Ensure that smart contract functionality operates as intended.

In summary, during the assessment **Halborn** identified several areas for improvement to reduce both the likelihood and impact of potential risks, which were mostly addressed by the **Aragon team**. The primary suggestions included:

- **Modify _deployStrategy() to pass the correct plugin address.**
- **Restrict strategy deployment to only be called by registered plugins.**
- **Implement a two-phase deployment process to prevent front-running.**
- **Add explicit validation to enforce future oriented time bounds.**
- **Introduce a maximum batch size constant and enforce it at the start of the function.**
- **Implement an emergency withdrawal mechanism that allows recovery of unclaimed tokens.**
- **Implement a permissioned registration system with trusted authorities.**
- **Implement enhanced token validation that applies universally to all campaigns.**
- **Remove the unused error If DAO mismatch validation is not needed.**
- **Validate the vault during setup which ensures that only compatible vaults can be configured for campaigns.**
- **Add validation to ensure the _metadata string is not empty.**

3. Test Approach And Methodology

Halborn performed a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is essential to uncover flaws in logic, process, and implementation, automated testing techniques enhance coverage of smart contracts and can quickly identify issues that do not follow security best practices.

The following phases and associated tools were used throughout the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual code review and walkthrough of the smart contracts to identify potential logic issues.
- Manual testing of all core functions, including **createCampaign**, claim to validate expected behavior and identify edge-case vulnerabilities.
- Local testing to simulate contract interactions and validate functional and security assumptions.
- Local deployment and testing with Foundry.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: [osx-capital-distributor](#)

(b) Assessed Commit ID: [870c8f8](#)

(c) Items in scope:

- src/CapitalDistributorPlugin.sol
- src/CapitalDistributorPluginSetup.sol
- src/allocatorStrategies/AllocatorStrategyBase.sol
- src/allocatorStrategies/MerkleDistributorStrategy.sol
- src/factories/ActionEncoderFactory.sol
- src/factories/AllocatorStrategyFactory.sol
- src/factories/FactoryBase.sol
- src/interfaces/IActionEncoderFactory.sol
- src/interfaces/IAllocatorStrategy.sol
- src/interfaces/IAllocatorStrategyFactory.sol
- src/interfaces/IPayoutActionEncoder.sol
- src/payoutActionEncoders/PayoutActionEncoderBase.sol
- src/payoutActionEncoders/VaultDepositPayoutActionEncoder.sol
- src/utils/RatioUtils.sol

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- [6d7b71b](#)
- [c62bfd6](#)

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW



SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
PLUGIN ADDRESS IN STRATEGY DEPLOYMENT BREAKS MERKLE ROOT UPDATES	HIGH	SOLVED - 10/22/2025
PRE-DEPLOYED STRATEGY OWNERSHIP MISMATCH BREAKS CAMPAIGN CREATION	MEDIUM	SOLVED - 10/22/2025
FRONT-RUNNING STRATEGY DEPLOYMENT CAUSES CAMPAIGN CREATION DOS	MEDIUM	SOLVED - 10/22/2025
CAMPAIGNS CAN BE CREATED WITH END TIME IN THE PAST	LOW	SOLVED - 10/22/2025
FUND LOCK DUE TO CAMPAIGN STATE MANAGEMENT	LOW	RISK ACCEPTED - 10/23/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
MALICIOUS ACTION ENCODER REGISTRATION ENABLES TOKEN THEFT	LOW	RISK ACCEPTED - 10/23/2025
INSUFFICIENT TOKEN VALIDATION IN CAMPAIGN CREATION	LOW	RISK ACCEPTED - 10/23/2025
UNUSED CUSTOM ERROR DAOMISMATCH	INFORMATIONAL	SOLVED - 10/22/2025
UNCHECKED ARRAY ITERATION IN BATCHCLAIMCAMPAIGNPAYOUT CAN CAUSE DOS	INFORMATIONAL	SOLVED - 10/22/2025
MISSING VAULT INTERFACE VALIDATION CAUSES SILENT CLAIM FAILURES	INFORMATIONAL	ACKNOWLEDGED - 10/23/2025
MISSING METADATA VALIDATION IN REGISTERACTIONENCODER AND REGISTERSTRATEGYTYPE	INFORMATIONAL	ACKNOWLEDGED - 10/23/2025

7. FINDINGS & TECH DETAILS

7.1 PLUGIN ADDRESS IN STRATEGY DEPLOYMENT BREAKS MERKLE ROOT UPDATES

// HIGH

Description

The `AllocatorStrategyFactory` contract incorrectly passes `msg.sender` (the deploying account) as the plugin address during strategy initialization, instead of passing the actual `CapitalDistributorPlugin` contract address. This causes the strategy to store an incorrect plugin address, breaking the merkle root update functionality which requires calling `isCampaignPaused()` on the plugin contract.

This vulnerability renders deployed merkle strategies permanently non-updatable, as the stored plugin address points to an EOA that cannot execute the required `CapitalDistributorPlugin::isCampaignPaused()` call.

Copy Code

```
171 | // Check if campaign is paused (not active, or ended)
172 | // The reason for this is so it's safe to take snapshots
173 | if (!CapitalDistributorPlugin(plugin).isCampaignPaused(_campaignId)) {
174 |     revert CampaignNotPaused(_campaignId);
```

The Factory contract stores EOA address instead of actual plugin address.

Copy Code

```
43 | plugin = _plugin;
44 | strategyId = _strategyId;
```

The `MerkleDistributorStrategy` is designed to distribute tokens using merkle trees, where eligible recipients can claim tokens by providing valid merkle proofs. The strategy should be deployed by a `CapitalDistributorPlugin` and maintain a reference to it for campaign management operations. The `MerkleDistributorStrategy::updateCampaignMerkleRoot()` function is intended to allow the DAO to update merkle roots only when campaigns are paused, ensuring safe snapshot operations.

Exploitation Steps

- EOA calls factory deployment: An EOA calls `AllocatorStrategyFactory.deployStrategy()` to create a new merkle strategy.
- Incorrect plugin address stored: The factory passes the EOA address as the plugin parameter instead of the actual plugin contract.
- Strategy becomes non-updatable: When the DAO later attempts to call `updateCampaignMerkleRoot()`, the function tries to call `isCampaignPaused()` on the EOA address.
- Transaction reverts: The call fails because EOAs don't implement the `CapitalDistributorPlugin` interface, making the strategy permanently non-updatable.

Proof of Concept

The following PoC demonstrates that by passing `msg.sender` (an EOA) in strategy deployment via `AllocatorStrategyFactory` as the plugin address, the deployed strategy stores an invalid reference. As a result, functions like `updateCampaignMerkleRoot()` revert, rendering the strategy permanently unmanageable.

Copy Code

```
function testUpdateMerkleRootFailsDueToPluginAddress() public {
    // Setup: Create campaign with initial merkle root
    token.mint(address(createdDao), 10 ether);
    bytes32 initialRoot = merkleRoot;

    vm.startPrank(address(createdDao));

    allocatorStrategyFactory.deployStrategy(toBytes32("merkle-strategy"), createdDao, "");

    bytes memory metadata = "ipfs://mock-campaign-metadata";
    bytes memory allocatorDeploymentParams = "";

    uint256 campaignId = capitalDistributorPlugin.createCampaign(
        metadata,
        CapitalDistributorPlugin.StrategyConfig(
            toBytes32("merkle-strategy"), allocatorDeploymentParams, abi.encode(initialRoot)
        ),
        CapitalDistributorPlugin.PayoutConfig(IERC20(token), bytes32(0), metadata),
        CapitalDistributorPlugin.CampaignSettings(0, 0)
    );

    // Pause the campaign
    capitalDistributorPlugin.pauseCampaign(campaignId);

    // Verify campaign is paused
    CapitalDistributorPlugin.Campaign memory campaign = capitalDistributorPlugin.getCampaign(campaignId);
    assertEq(
        uint8(campaign.state), uint8(CapitalDistributorPlugin.CampaignState.PAUSED), "Campaign should be paused"
    );

    // Update merkle root on paused campaign should succeed
    bytes32 newRoot = keccak256("new-merkle-root");
```

```
bytes memory newRootData = abi.encode(newRoot);

// Call directly on strategy but from DAO context
vm.stopPrank();
vm.prank(address(createdDao));
vm.expectRevert();
MerkleDistributorStrategy(address(campaign.allocationStrategy)).updateCampaignMerkleRoot(
    campaignId, newRootData
);
}
```

The following test confirms that deploying a strategy with `msg.sender` (an EOA) as the plugin address results in an invalid plugin reference. This causes strategy-level functions such as `updateCampaignMerkleRoot()` to revert permanently, making the campaign unmanageable even by the DAO:

```
Ran 1 test suite in 97.95ms (9.02ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
yash1923@yashs-MacBook-Pro osx-capital-distributor-870c8f82098499b330f1486dcdccb044323861e7 % forge test --mt testUpdateMerkleRootFailsDueToPluginAddress
[+] Compiling...
No files changed, compilation skipped

Ran 1 test for tests/allocatorStrategies/MerkleDistributorStrategyTest.t.sol:MerkleDistributorStrategyTest
[PASS] testUpdateMerkleRootFailsDueToPluginAddress() (gas: 500327)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 9.02ms (186.83µs CPU time)

Ran 1 test suite in 98.77ms (9.02ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (7.5)

Recommendation

Modify `_deployStrategy()` to pass the correct plugin address.

Remediation Comment

SOLVED: The Aragon team resolved the issue by introducing an `onlyCapitalDistributorPlugin` modifier in the `AllocatorStrategyFactory` contract. This modifier verifies that only valid `CapitalDistributorPlugin` contracts, identified through ERC-165 interface support for `ICapitalDistributorPlugin`, can deploy strategies.

Remediation Hash

<https://github.com/aragon/osx-capital-distributor/commit/6d7b71b928fc88efdc4add132d0ecdf62d60e15a>

7.2 PRE-DEPLOYED STRATEGY OWNERSHIP MISMATCH BREAKS CAMPAIGN CREATION

// MEDIUM

Description

The `CapitalDistributorPlugin.sol::createCampaign()` function attempts to call `ActionEncoder::setupCampaign()` on pre-deployed action encoders/strategies; because those contracts may be owned by the deploying EOA rather than the plugin, the owner-only check on `setupCampaign()` can cause the call to be rejected, allowing campaign creation to revert and blocking the use of those pre-deployed strategies.

When strategies or action encoders are deployed independently using `deployStrategy()` before campaign creation, they are owned by the deploying EOA rather than the `CapitalDistributorPlugin`. Subsequently, when `createCampaign()` attempts to call `setupCampaign()` on these pre-deployed contracts, the call fails because the plugin is not the owner, causing the entire campaign creation to revert.

The `msg.sender` is plugin, but owner is deploying EOA so it would revert.

 Copy Code

```
119 | function setAllocationCampaign(uint256 _campaignId, bytes calldata _auxData) public virtual override {
120 |     if (msg.sender != owner()) {
121 |         revert OnlyDAOAllowed(msg.sender);
122 |     }
123 |     // Campaign setup logic...
124 | }
```

The `CapitalDistributorPlugin` is designed to create distribution campaigns that utilize various allocation strategies and action encoders. The plugin should be able to set up campaigns on both newly deployed and existing strategy contracts. The `setupCampaign()` function is protected by an owner-only modifier to ensure only authorized entities can configure campaigns. However, the current ownership model creates a mismatch when contracts are pre-deployed.

Exploitation Steps

- EOA pre-deploys strategy: An EOA calls `AllocatorStrategyFactory.deployStrategy()` to deploy a strategy contract.
- Strategy ownership set to EOA: The deployed strategy's owner is set to the deploying EOA address.
- Plugin attempts campaign creation: The `CapitalDistributorPlugin` calls `createCampaign()` with parameters that would use the pre-deployed strategy.
- Setup call fails: When the plugin calls `setupCampaign()` on the pre-deployed strategy, the `onlyOwner` modifier rejects the call.
- Campaign creation reverts: The entire campaign creation transaction fails, preventing the use of that strategy.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (5.0)

Recommendation

Restrict strategy deployment to only be called by registered plugins, ensuring proper ownership from deployment.

Remediation Comment

SOLVED: The Aragon team resolved the issue by adding `msg.sender` in `_computeDeploymentId()`.

Remediation Hash

<https://github.com/aragon/osx-capital-distributor/commit/c62bfd6aef23b1efb3e9693c0334e999398cf611>

7.3 FRONT-RUNNING STRATEGY DEPLOYMENT CAUSES CAMPAIGN CREATION DOS

// MEDIUM

Description

The `CapitalDistributorPlugin.sol::createCampaign()` function attempts to deploy or reuse a strategy contract via `getOrCreateDeployStrategy()`, followed by calling `ActionEncoder::setupCampaign()` on the returned strategy. The problem arises because the deployment address is deterministically computed from predictable parameters (`_strategyId`, `dao()`, and `_deploymentParams`). This allows a malicious frontrunner to pre-deploy a strategy contract with identical parameters but under their own ownership.

When the original transaction from the plugin executes, it sees that the strategy is already deployed (at the expected address) and skips deployment. However, the plugin is not the owner of this contract. As a result, when it calls `setupCampaign()` on that strategy, the call fails due to the `onlyOwner` check, the plugin is not authorized to configure the strategy.

The `getOrCreateDeployStrategy()` returns attacker-deployed strategy.

Copy Code

```
317 | address strategy = factory.getOrCreateDeployStrategy(_strategyId, dao(), _deploymentParams);
318 |
319 | try actionEncoder.setupCampaign(id, _payout.actionEncoderInitData) { }
320 | catch {
321 |     // Campaign setup failed
322 | }
```

While the strategy contract is deployed, it is effectively poisoned because the plugin cannot use it. Since the plugin assumes it can reuse or deploy a usable strategy, it does not fall back to deploying a new one with a different address or ownership. This causes the `createCampaign()` transaction to revert, resulting in a denial-of-service for campaign creation.

Exploitation Steps

1. Attacker monitors mempool: Malicious EOA monitors pending `createCampaign()` transactions to extract deployment parameters.
2. Front-run deployment: Attacker calls `AllocatorStrategyFactory.deployStrategy()` with identical parameters using higher gas price.
3. Strategy deployed with wrong owner: The strategy gets deployed with the attacker's EOA as owner instead of the plugin.
4. Original transaction executes: The plugin's `createCampaign()` calls `getOrCreateDeployStrategy()` which returns the existing (attacker-owned) strategy.
5. Setup call fails: When plugin attempts `setupCampaign()`, the `onlyOwner` check fails because `msg.sender` (plugin) is not the owner (attacker's EOA).
6. Campaign creation reverts: The entire campaign creation fails, causing denial of service.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (5.0)

Recommendation

Implement a two-phase deployment process to prevent front-running or Restrict deployment to registered plugins.

Remediation Comment

SOLVED: The Aragon team resolved the issue by modifying the `_computeDeploymentId()` function to include `msg.sender` for accurate address computation and added the `deployStrategy()` and `deployActionEncoder()` functions to the `CapitalDistributorPlugin` contract.

Remediation Hash

<https://github.com/aragon/osx-capital-distributor/commit/c62bfd6aef23b1efb3e9693c0334e999398cf611>

7.4 CAMPAIGNS CAN BE CREATED WITH END TIME IN THE PAST

// LOW

Description

The `CapitalDistributorPlugin::createCampaign()` function only validates that `startTime < endTime` when both are set, but it does not check whether the provided times are in the future. This allows campaigns to be created with end times already in the past, making them immediately expired and unusable.

Copy Code

```
285 | function createCampaign(  
286 |     bytes calldata _metadataURI,  
287 |     StrategyConfig calldata _strategy,  
288 |     PayoutConfig calldata _payout,  
289 |     CampaignSettings calldata _settings  
290 | )  
291 |     external  
292 |     auth(CAMPAIGN_MANAGER_PERMISSION_ID)  
293 |     returns (uint256 id)  
294 | {  
295 |     // Input validation  
296 |     if (_metadataURI.length == 0) {  
297 |         revert EmptyMetadataURI();  
298 |     }  
299 |  
300 |     if (address(_payout.token) == address(0)) {  
301 |         revert ZeroAddress("_token");  
302 |     }  
303 |     if (_settings.startTime > 0 && _settings.endTime > 0 && _settings.startTime >= _settings.endTime) {  
304 |         revert InvalidTimeBounds();  
305 |     }  
306 | }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Add explicit validation to enforce future oriented time bounds.

Remediation Comment

SOLVED: The Aragon team resolved the issue by adding a validation of `endTime` with `block.timestamp` in `createCampaign()`.

Remediation Hash

<https://github.com/aragon/osx-capital-distributor/commit/6d7b71b928fc88efdc4add132d0ecdf62d60e15a>

7.5 FUND LOCK DUE TO CAMPAIGN STATE MANAGEMENT

// LOW

Description

The `CapitalDistributorPlugin::resumeCampaign()` function contains a flawed time boundary check that prevents paused campaigns from being resumed if their end time has passed, creating a permanent lock state where funds cannot be accessed by users or recovered by administrators.

Copy Code

```
683 | function resumeCampaign(uint256 _campaignId) external auth(CAMPAIGN_MANAGER_PERMISSION_ID) {
684 |     _requireCampaignExists(_campaignId);
685 |     Campaign storage campaign = campaigns[_campaignId];
686 |
687 |     if (campaign.state != CampaignState.PAUSED) {
688 |         revert InvalidStateTransition(_campaignId, campaign.state, CampaignState.ACTIVE);
689 |     }
690 |
691 |     // Prevent resuming campaigns that have already ended due to time
692 |     if (campaign.endTime > 0 && block.timestamp >= campaign.endTime) {
693 |         revert CampaignOutsideTimeBounds(_campaignId, block.timestamp, campaign.startTime, campaign.endTime);
694 |     }
695 |
696 |     campaign.state = CampaignState.ACTIVE;
697 |     emit CampaignResumed(_campaignId);
698 | }
```

The `CapitalDistributorPlugin` manages token distribution campaigns with three states: `ACTIVE`, `PAUSED`, and `ENDED`. Campaigns can have optional time boundaries (`startTime` and `endTime`). The pause/resume mechanism is designed to allow temporary suspension of claims while maintaining the ability to restore normal operation. However, the current implementation creates scenarios where campaigns become permanently inaccessible without any recovery mechanism.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (2.5)

Recommendation

To address the fund lock vulnerability, implement an emergency withdrawal mechanism that allows recovery of unclaimed tokens after campaign expiration.

Remediation Comment

RISK ACCEPTED: The **Aragon team** made a business decision to accept the risk of this finding and not alter the contracts.

7.6 MALICIOUS ACTION ENCODER REGISTRATION ENABLES TOKEN THEFT

// LOW

Description

The `ActionEncoderFactory.sol::registerActionEncoder()` function allows registration of arbitrary contracts as action encoder implementations, as long as they support the `IPayoutActionEncoder` interface. This can be exploited by attackers to register malicious encoder contracts that technically conform to the interface but perform unauthorized or malicious logic (e.g., redirecting funds). Since the factory performs no validation of *what* the encoder does beyond the interface check, this can result in token theft when such encoders are later used in campaigns.

 Copy Code

```
33 function registerActionEncoder(  
34     bytes32 _encoderId,  
35     address _encoderImplementation,  
36     string calldata _metadata  
37 )  
38     external  
39 {  
40     // Validate basic requirements first  
41     if (_encoderId == bytes32(0)) {  
42         revert EmptyTypeId();  
43     }  
44     if (_encoderImplementation == address(0)) {  
45         revert InvalidImplementation(_encoderImplementation, "Implementation address cannot be zero");  
46     }  
47     if (registeredTypes[_encoderId].implementation != address(0)) {  
48         revert AlreadyRegistered(_encoderId);  
49     }  
50     if (_encoderImplementation.code.length == 0) {  
51         revert InvalidImplementation(_encoderImplementation, "Implementation must be a deployed contract");  
52     }  
53  
54     // Validate that the implementation supports the IPayoutActionEncoder interface  
55     try IERC165(_encoderImplementation).supportsInterface(type(IPayoutActionEncoder).interfaceId) returns (   
56         bool supported  
57     ) {  
58         if (!supported) {  
59             revert InvalidImplementation(  
60                 _encoderImplementation, "Implementation must support IPayoutActionEncoder interface"  
61             );  
62         }  
63     } catch {  
64         revert InvalidImplementation(  
65             _encoderImplementation, "Implementation must support IPayoutActionEncoder interface"  
66         );  
67     }  
68  
69     // Register the type  
70     registeredTypes[_encoderId] = RegisteredType({ implementation: _encoderImplementation, metadata: _metadata });  
71     emit TypeRegistered(_encoderId, _encoderImplementation, _metadata, msg.sender);  
72     emit ActionEncoderTypeRegistered(_encoderId, _encoderImplementation, _metadata);  
73 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (2.5)

Recommendation

Implement a permissioned registration system with trusted authorities.

Remediation Comment

RISK ACCEPTED: The Aragon team made a business decision to accept the risk of this finding and not alter the contracts.

7.7 INSUFFICIENT TOKEN VALIDATION IN CAMPAIGN CREATION

// LOW

Description

The `CapitalDistributorPlugin.sol::_validateTokenBehavior()` function uses inadequate token validation that can be bypassed by malicious tokens designed to appear legitimate during validation but fail silently during actual transfers. Additionally, the validation is completely skipped when action encoders are used, creating a critical security gap. The `CapitalDistributorPlugin` allows Aragon DAOs to create token distribution campaigns where recipients can claim allocated tokens.

Copy Code

```
370 | function _validateTokenBehavior(IERC20 _token) internal view {
371 |     (bool success,) =
372 |         address(_token).staticcall(abi.encodeCall(IERC20.transferFrom, (address(0), address(this), 1)));
373 |     if (success) {
374 |         // If the call succeeded, for whatever reason, it should revert
375 |         revert InvalidToken(address(_token));
376 |     }
377 | }
```

The `_validateTokenBehavior()` function is intended to prevent the DAO from using tokens that don't properly revert on invalid transfers, which could lead to silent failures where users believe they received tokens but didn't. This validation is crucial because the DAO executes token transfers on behalf of claimants, and silent failures would break the entire distribution mechanism.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

The contract should implement enhanced token validation that applies universally to all campaigns and modify the `_validateTokenBehavior()` function to test multiple invalid transfer scenarios instead of relying on a single check.

Remediation Comment

RISK ACCEPTED: The Aragon team made a business decision to accept the risk of this finding and not alter the contracts.

7.8 UNUSED CUSTOM ERROR DAOMISMATCH

// INFORMATIONAL

Description

The `CapitalDistributorPlugin` contract defines a custom error `DAOMismatch(address expected, address provided);`, intended to signal a mismatch between the expected DAO address and the one provided (presumably during strategy setup or execution). However, this error is never used anywhere in the contract, making it redundant and potentially misleading:

Copy Code

```
179 | error DAOMismatch(address expected, address provided);
```

BVSS

AO:A/AC:M/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (1.7)

Recommendation

If DAO mismatch validation is not needed, remove the unused error.

Remediation Comment

SOLVED: The Aragon team resolved the issue by removing `DAOMismatch` custom error.

Remediation Hash

<https://github.com/aragon/osx-capital-distributor/commit/6d7b71b928fc88efdc4add132d0ecdf62d60e15a>

7.9 UNCHECKED ARRAY ITERATION IN BATCHCLAIMCAMPAIGNPAYOUT CAN CAUSE DOS

// INFORMATIONAL

Description

The `CapitalDistributorPlugin::batchClaimCampaignPayout()` function iterates over user-supplied arrays (`_campaignIds`, `_recipients`, `_strategiesAuxData`, `_encodersAuxData`) without enforcing a maximum length. Users can supply excessively large arrays, causing the transaction to run out of gas and revert.

Copy Code

```
726 function batchClaimCampaignPayout(  
727     uint256[] calldata _campaignIds,  
728     address[] calldata _recipients,  
729     bytes[] calldata _strategiesAuxData,  
730     bytes[] calldata _encodersAuxData  
731 )  
732     external  
733     returns (uint256[] memory amounts)  
734 {  
735     uint256 length = _campaignIds.length;  
736     if (length != _recipients.length || length != _strategiesAuxData.length || length != _encodersAuxData.length) {  
737         revert ArrayLengthMismatch();  
738     }  
739     amounts = new uint256[](length);  
740     for (uint256 i = 0; i < length; ++i) {  
741         amounts[i] =  
742             claimCampaignPayout(_campaignIds[i], _recipients[i], _strategiesAuxData[i], _encodersAuxData[i]);  
743     }  
744     return amounts;  
745 }  
746  
747  
748 }
```

This function is intended to optimize gas costs by allowing multiple payout claims in a single call. However, without a batch size limit, the loop becomes a denial-of-service (DoS) vector, large inputs can always exceed block gas limits, making the function unreliable in production.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (1.7)

Recommendation

Introduce a maximum batch size constant and enforce it at the start of the function.

Remediation Comment

SOLVED: The Aragon team resolved the issue by introducing `MAX_BATCH_SIZE` in `batchClaimCampaignPayout()`.

Remediation Hash

<https://github.com/aragon/osx-capital-distributor/commit/6d7b71b928fc88efdc4add132d0ecdf62d60e15a>

7.10 MISSING VAULT INTERFACE VALIDATION CAUSES SILENT CLAIM FAILURES

// INFORMATIONAL

Description

The `VaultDepositPayoutActionEncoder` assumes that external vaults implement a specific interface `deposit(uint256 amount, address recipient)`. However, no validation is performed during `ActionEncoder::setupCampaign()` to confirm this. If a vault uses a different signature (e.g., `deposit(address,uint256)`) or reverts under certain conditions, payout execution will fail at runtime, leaving users unable to claim their allocations.

Copy Code

```
57 function buildActions(  
58     IERC20 _token,  
59     address _recipient,  
60     uint256 _amount,  
61     address  
62     uint256 _campaignId,  
63     bytes memory  
64 )  
65     external  
66     view  
67     override  
68     returns (Action[] memory actions)  
69 {  
70     if (_amount == 0) {  
71         revert AmountCannotBeZero();  
72     }  
73  
74     address vaultAddress = campaignVaults[_campaignId];  
75     if (vaultAddress == address(0)) {  
76         revert VaultNotSetForCampaign(_campaignId);  
77     }  
78  
79     actions = new Action[](2);  
80     actions[0] = Action({ to: address(_token), value: 0, data: abi.encodeCall(IERC20.approve, (vaultAddress, _amount)) });  
81  
82     actions[1] = Action({ to: vaultAddress, value: 0, data: abi.encodeCall(IVault.deposit, (_amount, _recipient)) });  
83  
84     return actions;  
85 }
```

BVSS

AO:A/AC:M/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (1.7)

Recommendation

Validate the vault during setup which ensures that only compatible vaults can be configured for campaigns.

Remediation Comment

ACKNOWLEDGED: The Aragon team acknowledged this finding, also mentioning that:

DAO is responsible of validating the encoder works with their vault or tokens.

7.11 MISSING METADATA VALIDATION IN REGISTERACTIONENCODER AND REGISTERSTRATEGYTYPE

// INFORMATIONAL

Description

The `ActionEncoderFactory::registerStrategyType()` function (and similarly in related factories like `AllocatorStrategyFactory::registerStrategyType()`) accepts a `string calldata _metadata` parameter, but does **not perform any validation** on it. This opens up the potential for empty, meaningless, or malformed metadata being registered, which may hinder off-chain indexing, front-end rendering, or future integrations that rely on this metadata field being meaningful and non-empty.

Copy Code

```
33 | function registerStrategyType(  
34 |     bytes32 _strategyId,  
35 |     address _strategyImplementation,  
36 |     string calldata _metadata,  
37 |     address _feeRecipient,  
38 |     uint32 _feeBasisPoints  
39 | ) external {  
40 |  
41 |     ... No validation on `_metadata`  
42 | }
```

BVSS

AO:A/AC:M/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

Add validation to ensure the `_metadata` string is not empty.

Remediation Comment

ACKNOWLEDGED: The **Aragon team** acknowledged this finding, also mentioning that:

Metadata can be empty. We don't plan to enforce any format.

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project’s integrity and addressing potential vulnerabilities introduced by code modifications.